Module 10: Review and Final Project Work

Learning Objectives

- Comprehensive review of course materials
- Integration of all learned concepts
- Project planning and execution
- Code organization and best practices
- Documentation and presentation skills

Topics Covered

1. Course material review and integration
2. Project ideation and planning
3. Data collection and preparation
4. Exploratory data analysis
5. Model development and evaluation
6. Results interpretation and visualization
7. Code organization and documentation
8. Version control with Git
9. Project presentation preparation
10. Best practices and code review
11. Future learning paths

Final Project Guidelines

- Choose a real-world dataset
- Apply complete data science workflow
- Include data cleaning, analysis, and modeling
- Create compelling visualizations
- Document your process and findings

1. Course Material Review and Integration

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import warnings
warnings.filterwarnings('ignore')
```

```
# Course Review: Key Concepts Integration
print("=== PYTHON COURSE REVIEW ===")
print("\n1. PYTHON FUNDAMENTALS:")
print("   - Variables, data types, control flow")
print("   - Functions, classes, object-oriented programming")
print("   - File handling, exception handling")

print("\n2. DATA MANIPULATION:")
print("   - NumPy: Numerical computing with arrays")
print("   - Pandas: Data analysis with Series and DataFrames")
print("   - Data cleaning, preprocessing, and transformation")

print("\n3. DATA VISUALIZATION:")
print("   - Matplotlib: Basic plotting and customization")
print("   - Seaborn: Statistical data visualization")
print("   - Creating meaningful charts and graphs")

print("\n4. MACHINE LEARNING:")
print("   - Supervised learning: Classification and Regression")
print("   - Unsupervised learning: Clustering and Dimensionality Reductio
print("   - Model evaluation and validation")

print("\n5. BEST PRACTICES:")
print("   - Code organization and documentation")
print("   - Version control with Git")
print("   - Data science project workflow")

# Sample data for demonstration
np.random.seed(42)
data = {
    'age': np.random.randint(18, 65, 100),
    'income': np.random.normal(50000, 15000, 100),
    'education': np.random.choice(['High School', 'Bachelor', 'Master', '
    'experience': np.random.randint(0, 30, 100),
    'salary': np.random.normal(60000, 20000, 100)
}

df = pd.DataFrame(data)
print(f"\nSample Dataset Shape: {df.shape}")
print(f"Dataset Info:")
print(df.info())
```

2. Complete Data Science Workflow Example

3. Comprehensive Python Fundamentals Review

3.1 Data Types and Structures

In [ ]:
```
# Python Fundamentals Review - Data Types and Structures
print("=== PYTHON FUNDAMENTALS REVIEW ===")

# 1. Basic Data Types
print("\n1. BASIC DATA TYPES:")
print("   - Numbers: int, float, complex")
print("   - Text: str")
print("   - Boolean: bool")
```

```python
print("   - None: NoneType")

# Examples
age = 25                    # int
height = 5.9                # float
name = "Alice"              # str
is_student = True           # bool
data = None                 # NoneType

print(f"\nExamples:")
print(f"age = {age} (type: {type(age)})")
print(f"height = {height} (type: {type(height)})")
print(f"name = '{name}' (type: {type(name)})")
print(f"is_student = {is_student} (type: {type(is_student)})")
print(f"data = {data} (type: {type(data)})")

# 2. Collections
print("\n2. COLLECTIONS:")
print("   - Lists: Ordered, mutable, allows duplicates")
print("   - Tuples: Ordered, immutable, allows duplicates")
print("   - Sets: Unordered, mutable, no duplicates")
print("   - Dictionaries: Key-value pairs, mutable")

# Examples
fruits = ['apple', 'banana', 'orange']          # list
coordinates = (10, 20)                          # tuple
unique_numbers = {1, 2, 3, 4, 5}               # set
person = {'name': 'Alice', 'age': 25, 'city': 'NYC'}  # dict

print(f"\nExamples:")
print(f"fruits = {fruits} (type: {type(fruits)})")
print(f"coordinates = {coordinates} (type: {type(coordinates)})")
print(f"unique_numbers = {unique_numbers} (type: {type(unique_numbers)})"
print(f"person = {person} (type: {type(person)})")

# 3. String Operations
print("\n3. STRING OPERATIONS:")
text = "Hello, World!"
print(f"Original: {text}")
print(f"Upper: {text.upper()}")
print(f"Lower: {text.lower()}")
print(f"Split: {text.split(', ')}")
print(f"Replace: {text.replace('World', 'Python')}")
print(f"Length: {len(text)}")

# 4. List Comprehensions
print("\n4. LIST COMPREHENSIONS:")
numbers = [1, 2, 3, 4, 5]
squares = [x**2 for x in numbers]
even_squares = [x**2 for x in numbers if x % 2 == 0]

print(f"Numbers: {numbers}")
print(f"Squares: {squares}")
print(f"Even squares: {even_squares}")

# 5. Dictionary Comprehensions
print("\n5. DICTIONARY COMPREHENSIONS:")
word_lengths = {word: len(word) for word in ['apple', 'banana', 'cherry']}
print(f"Word lengths: {word_lengths}")
```

```python
# 6. Functions and Classes
print("\n6. FUNCTIONS AND CLASSES:")

def calculate_area(length, width):
    """Calculate the area of a rectangle."""
    return length * width

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def greet(self):
        return f"Hello, I'm {self.name} and I'm {self.age} years old."

# Function example
area = calculate_area(5, 3)
print(f"Area of 5x3 rectangle: {area}")

# Class example
person = Person("Alice", 25)
print(f"Person greeting: {person.greet()}")

# 7. Error Handling
print("\n7. ERROR HANDLING:")
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Error: Division by zero!")
finally:
    print("This always executes")

# 8. File Operations
print("\n8. FILE OPERATIONS:")
# Writing to file
with open('sample_data.txt', 'w') as f:
    f.write("Sample data for demonstration\n")
    f.write("This is line 2\n")

# Reading from file
with open('sample_data.txt', 'r') as f:
    content = f.read()
    print(f"File content:\n{content}")

print("\n=== END OF PYTHON FUNDAMENTALS REVIEW ===")
```

4. NumPy Comprehensive Review

4.1 Array Creation and Operations

In [ ]:

```python
# NumPy Comprehensive Review
print("=== NUMPY COMPREHENSIVE REVIEW ===")

# 1. Array Creation
print("\n1. ARRAY CREATION:")
print("   - From lists/tuples")
print("   - Using built-in functions")
print("   - Special arrays (zeros, ones, identity)")
```

```python
# Examples
arr1 = np.array([1, 2, 3, 4, 5])                    # 1D array
arr2 = np.array([[1, 2, 3], [4, 5, 6]])             # 2D array
arr3 = np.zeros((3, 4))                             # Array of zeros
arr4 = np.ones((2, 3))                              # Array of ones
arr5 = np.eye(3)                                    # Identity matrix
arr6 = np.arange(0, 10, 2)                          # Range array
arr7 = np.linspace(0, 1, 5)                         # Linear space
arr8 = np.random.random((2, 3))                     # Random array

print(f"\nExamples:")
print(f"1D array: {arr1}")
print(f"2D array:\n{arr2}")
print(f"Zeros array:\n{arr3}")
print(f"Ones array:\n{arr4}")
print(f"Identity matrix:\n{arr5}")
print(f"Range array: {arr6}")
print(f"Linear space: {arr7}")
print(f"Random array:\n{arr8}")

# 2. Array Properties
print("\n2. ARRAY PROPERTIES:")
print(f"Shape: {arr2.shape}")
print(f"Size: {arr2.size}")
print(f"Data type: {arr2.dtype}")
print(f"Dimensions: {arr2.ndim}")
print(f"Memory usage: {arr2.nbytes} bytes")

# 3. Array Indexing and Slicing
print("\n3. ARRAY INDEXING AND SLICING:")
matrix = np.array([[1, 2, 3, 4],
                   [5, 6, 7, 8],
                   [9, 10, 11, 12]])

print(f"Original matrix:\n{matrix}")
print(f"Element at [1,2]: {matrix[1, 2]}")
print(f"First row: {matrix[0, :]}")
print(f"Second column: {matrix[:, 1]}")
print(f"Submatrix [1:3, 1:3]:\n{matrix[1:3, 1:3]}")

# 4. Array Operations
print("\n4. ARRAY OPERATIONS:")
a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])

print(f"Array a: {a}")
print(f"Array b: {b}")
print(f"Addition: {a + b}")
print(f"Subtraction: {a - b}")
print(f"Multiplication: {a * b}")
print(f"Division: {a / b}")
print(f"Power: {a ** 2}")
print(f"Square root: {np.sqrt(a)}")

# 5. Mathematical Functions
print("\n5. MATHEMATICAL FUNCTIONS:")
data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

print(f"Data: {data}")
```

```python
print(f"Sum: {np.sum(data)}")
print(f"Mean: {np.mean(data):.2f}")
print(f"Median: {np.median(data)}")
print(f"Standard deviation: {np.std(data):.2f}")
print(f"Variance: {np.var(data):.2f}")
print(f"Min: {np.min(data)}")
print(f"Max: {np.max(data)}")
print(f"Argmin: {np.argmin(data)}")
print(f"Argmax: {np.argmax(data)}")

# 6. Array Reshaping
print("\n6. ARRAY RESHAPING:")
original = np.arange(12)
print(f"Original (1D): {original}")

reshaped = original.reshape(3, 4)
print(f"Reshaped (3x4):\n{reshaped}")

flattened = reshaped.flatten()
print(f"Flattened: {flattened}")

# 7. Broadcasting
print("\n7. BROADCASTING:")
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])
scalar = 10

print(f"Matrix:\n{matrix}")
print(f"Scalar: {scalar}")
print(f"Matrix + scalar:\n{matrix + scalar}")

# 8. Linear Algebra
print("\n8. LINEAR ALGEBRA:")
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print(f"Matrix A:\n{A}")
print(f"Matrix B:\n{B}")
print(f"Matrix multiplication:\n{np.dot(A, B)}")
print(f"Transpose of A:\n{A.T}")
print(f"Determinant of A: {np.linalg.det(A):.2f}")
print(f"Inverse of A:\n{np.linalg.inv(A)}")

# 9. Statistical Operations
print("\n9. STATISTICAL OPERATIONS:")
data = np.random.normal(100, 15, 1000)  # Normal distribution

print(f"Sample size: {len(data)}")
print(f"Mean: {np.mean(data):.2f}")
print(f"Std: {np.std(data):.2f}")
print(f"25th percentile: {np.percentile(data, 25):.2f}")
print(f"75th percentile: {np.percentile(data, 75):.2f}")

# 10. Performance Comparison
print("\n10. PERFORMANCE COMPARISON:")
import time

size = 1000000

# Python list
```

```
python_list = list(range(size))
start_time = time.time()
python_sum = sum(python_list)
python_time = time.time() - start_time

# NumPy array
numpy_array = np.arange(size)
start_time = time.time()
numpy_sum = np.sum(numpy_array)
numpy_time = time.time() - start_time

print(f"Python list sum: {python_sum} (time: {python_time:.4f}s)")
print(f"NumPy array sum: {numpy_sum} (time: {numpy_time:.4f}s)")
print(f"NumPy is {python_time/numpy_time:.1f}x faster!")

print("\n=== END OF NUMPY REVIEW ===")
```

5. Advanced Pandas Operations Review

5.1 Series and DataFrame Fundamentals

In [ ]:

```
# Advanced Pandas Operations Review
print("=== ADVANCED PANDAS OPERATIONS REVIEW ===")

# 1. Series Operations
print("\n1. SERIES OPERATIONS:")
print("   - Creation, indexing, and selection")
print("   - Mathematical operations")
print("   - String operations")
print("   - Date/time operations")

# Create sample data
np.random.seed(42)
dates = pd.date_range('2023-01-01', periods=10, freq='D')
sales = pd.Series(np.random.randint(1000, 5000, 10), index=dates, name='S
products = pd.Series(['Laptop', 'Phone', 'Tablet', 'Laptop', 'Phone',
                      'Tablet', 'Laptop', 'Phone', 'Tablet', 'Laptop'])

print(f"\nSales Series:")
print(sales)
print(f"\nProducts Series:")
print(products)

# Series operations
print(f"\nSales statistics:")
print(f"Mean: {sales.mean():.2f}")
print(f"Sum: {sales.sum():.2f}")
print(f"Max: {sales.max()}")
print(f"Min: {sales.min()}")

# 2. DataFrame Creation and Manipulation
print("\n2. DATAFRAME CREATION AND MANIPULATION:")

# Create comprehensive dataset
data = {
    'Employee_ID': range(1, 11),
    'Name': ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve', 'Frank', 'Grace',
    'Department': ['IT', 'HR', 'Finance', 'IT', 'HR', 'Finance', 'IT', 'H
```

```
    'Salary': [75000, 65000, 80000, 70000, 60000, 85000, 72000, 68000, 90
    'Experience': [5, 3, 8, 4, 2, 10, 6, 4, 12, 7],
    'Performance': ['Excellent', 'Good', 'Excellent', 'Good', 'Average',
    'Hire_Date': pd.date_range('2020-01-01', periods=10, freq='M')
}

df = pd.DataFrame(data)
print(f"DataFrame shape: {df.shape}")
print(f"\nFirst 5 rows:")
print(df.head())

# 3. Advanced Indexing and Selection
print("\n3. ADVANCED INDEXING AND SELECTION:")

# Using loc and iloc
print(f"Using loc - first 3 rows, Name and Salary columns:")
print(df.loc[0:2, ['Name', 'Salary']])

print(f"\nUsing iloc - first 3 rows, columns 1 and 3:")
print(df.iloc[0:3, [1, 3]])

# Boolean indexing
high_salary = df[df['Salary'] > 75000]
print(f"\nHigh salary employees (>75000):")
print(high_salary[['Name', 'Department', 'Salary']])

# 4. GroupBy Operations
print("\n4. GROUPBY OPERATIONS:")

# Basic groupby
dept_stats = df.groupby('Department')['Salary'].agg(['mean', 'min', 'max'
print(f"Department salary statistics:")
print(dept_stats)

# Multiple aggregations
dept_performance = df.groupby(['Department', 'Performance']).size().unsta
print(f"\nPerformance by department:")
print(dept_performance)

# 5. Pivot Tables
print("\n5. PIVOT TABLES:")

# Create pivot table
pivot_table = df.pivot_table(
    values='Salary',
    index='Department',
    columns='Performance',
    aggfunc='mean',
    fill_value=0
)
print(f"Average salary by department and performance:")
print(pivot_table)

# 6. Merging and Joining
print("\n6. MERGING AND JOINING:")

# Create second DataFrame
bonus_data = {
    'Employee_ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Bonus': [5000, 3000, 8000, 4000, 2000, 10000, 6000, 3500, 12000, 700
```

```
}
bonus_df = pd.DataFrame(bonus_data)

# Inner join
merged_df = df.merge(bonus_df, on='Employee_ID', how='inner')
print(f"Merged DataFrame (first 5 rows):")
print(merged_df[['Name', 'Department', 'Salary', 'Bonus']].head())

# 7. Data Cleaning and Preprocessing
print("\n7. DATA CLEANING AND PREPROCESSING:")

# Create dataset with missing values
dirty_data = df.copy()
dirty_data.loc[2, 'Salary'] = np.nan
dirty_data.loc[5, 'Department'] = None
dirty_data.loc[8, 'Experience'] = np.nan

print(f"Dataset with missing values:")
print(dirty_data.isnull().sum())

# Handle missing values
cleaned_data = dirty_data.copy()
cleaned_data['Salary'].fillna(cleaned_data['Salary'].mean(), inplace=True
cleaned_data['Department'].fillna('Unknown', inplace=True)
cleaned_data['Experience'].fillna(cleaned_data['Experience'].median(), in

print(f"\nAfter cleaning:")
print(cleaned_data.isnull().sum())

# 8. String Operations
print("\n8. STRING OPERATIONS:")

# String methods on Series
print(f"Name column operations:")
print(f"All names: {df['Name'].tolist()}")
print(f"Names starting with 'A': {df[df['Name'].str.startswith('A')]['Nam
print(f"Names containing 'a': {df[df['Name'].str.contains('a', case=False

# 9. Date/Time Operations
print("\n9. DATE/TIME OPERATIONS:")

# Set Hire_Date as index
df_time = df.set_index('Hire_Date')
print(f"DataFrame with date index:")
print(df_time.head())

# Date operations
print(f"\nDate operations:")
print(f"Year range: {df_time.index.year.min()} - {df_time.index.year.max(
print(f"Employees hired in 2020: {len(df_time[df_time.index.year == 2020]

# 10. Window Functions
print("\n10. WINDOW FUNCTIONS:")

# Rolling statistics
df['Salary_Rolling_Mean'] = df['Salary'].rolling(window=3, min_periods=1)
print(f"Salary with rolling mean:")
print(df[['Name', 'Salary', 'Salary_Rolling_Mean']].head())

# 11. Data Export/Import
```

```
print("\n11. DATA EXPORT/IMPORT:")

# Export to CSV
df.to_csv('employee_data.csv', index=False)
print(f"Data exported to employee_data.csv")

# Read from CSV
df_imported = pd.read_csv('employee_data.csv')
print(f"Imported data shape: {df_imported.shape}")

print("\n=== END OF PANDAS REVIEW ===")
```

6. Comprehensive Data Visualization Review

6.1 Matplotlib and Seaborn Fundamentals

In [ ]:
```
# Comprehensive Data Visualization Review
print("=== COMPREHENSIVE DATA VISUALIZATION REVIEW ===")

# Set up the plotting style
plt.style.use('default')
sns.set_palette("husl")

# 1. Matplotlib Fundamentals
print("\n1. MATPLOTLIB FUNDAMENTALS:")
print("   - Figure and axes management")
print("   - Basic plot types")
print("   - Customization and styling")

# Create sample data
np.random.seed(42)
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)
y3 = np.sin(x) * np.cos(x)

# Basic line plots
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.plot(x, y1, label='sin(x)', linewidth=2)
plt.plot(x, y2, label='cos(x)', linewidth=2)
plt.title('Basic Line Plot')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.3)

# 2. Different Plot Types
print("\n2. DIFFERENT PLOT TYPES:")

# Scatter plot
np.random.seed(42)
x_scatter = np.random.randn(100)
y_scatter = 2 * x_scatter + np.random.randn(100)

plt.subplot(2, 3, 2)
plt.scatter(x_scatter, y_scatter, alpha=0.6, c='red')
```

```
plt.title('Scatter Plot')
plt.xlabel('X')
plt.ylabel('Y')

# Histogram
data_hist = np.random.normal(0, 1, 1000)
plt.subplot(2, 3, 3)
plt.hist(data_hist, bins=30, alpha=0.7, color='green', edgecolor='black')
plt.title('Histogram')
plt.xlabel('Value')
plt.ylabel('Frequency')

# Bar plot
categories = ['A', 'B', 'C', 'D', 'E']
values = [23, 45, 56, 78, 32]
plt.subplot(2, 3, 4)
plt.bar(categories, values, color='orange')
plt.title('Bar Plot')
plt.xlabel('Category')
plt.ylabel('Value')

# Pie chart
sizes = [30, 25, 20, 15, 10]
labels = ['Category 1', 'Category 2', 'Category 3', 'Category 4', 'Catego
plt.subplot(2, 3, 5)
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
plt.title('Pie Chart')

# Box plot
data_box = [np.random.normal(0, 1, 100),
            np.random.normal(1, 1.5, 100),
            np.random.normal(2, 0.5, 100)]
plt.subplot(2, 3, 6)
plt.boxplot(data_box, labels=['Group 1', 'Group 2', 'Group 3'])
plt.title('Box Plot')
plt.ylabel('Value')

plt.tight_layout()
plt.show()

# 3. Seaborn Statistical Plots
print("\n3. SEABORN STATISTICAL PLOTS:")

# Create sample dataset
np.random.seed(42)
n_samples = 200
data = {
    'x': np.random.randn(n_samples),
    'y': 2 * np.random.randn(n_samples) + 1,
    'category': np.random.choice(['A', 'B', 'C'], n_samples),
    'value': np.random.exponential(2, n_samples)
}
df_viz = pd.DataFrame(data)

# Create subplots for Seaborn
fig, axes = plt.subplots(2, 3, figsize=(15, 10))

# Distribution plot
sns.histplot(data=df_viz, x='x', kde=True, ax=axes[0, 0])
axes[0, 0].set_title('Distribution Plot')
```

```python
# Scatter plot with regression
sns.scatterplot(data=df_viz, x='x', y='y', hue='category', ax=axes[0, 1])
sns.regplot(data=df_viz, x='x', y='y', scatter=False, ax=axes[0, 1])
axes[0, 1].set_title('Scatter Plot with Regression')

# Box plot
sns.boxplot(data=df_viz, x='category', y='value', ax=axes[0, 2])
axes[0, 2].set_title('Box Plot by Category')

# Violin plot
sns.violinplot(data=df_viz, x='category', y='value', ax=axes[1, 0])
axes[1, 0].set_title('Violin Plot by Category')

# Heatmap
correlation_data = df_viz[['x', 'y', 'value']].corr()
sns.heatmap(correlation_data, annot=True, cmap='coolwarm', center=0, ax=a
axes[1, 1].set_title('Correlation Heatmap')

# Pair plot (simplified version)
sns.scatterplot(data=df_viz, x='x', y='y', hue='category', ax=axes[1, 2])
axes[1, 2].set_title('Pair Plot (X vs Y)')

plt.tight_layout()
plt.show()

# 4. Advanced Customization
print("\n4. ADVANCED CUSTOMIZATION:")

# Create a professional-looking plot
fig, ax = plt.subplots(figsize=(10, 6))

# Generate time series data
dates = pd.date_range('2023-01-01', periods=365, freq='D')
values = np.cumsum(np.random.randn(365)) + 100

# Plot with custom styling
ax.plot(dates, values, linewidth=2, color='#2E86AB', label='Time Series')
ax.fill_between(dates, values, alpha=0.3, color='#2E86AB')

# Customize the plot
ax.set_title('Professional Time Series Plot', fontsize=16, fontweight='bo
ax.set_xlabel('Date', fontsize=12)
ax.set_ylabel('Value', fontsize=12)
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3)

# Format x-axis
ax.xaxis.set_major_formatter(plt.matplotlib.dates.DateFormatter('%Y-%m'))
ax.xaxis.set_major_locator(plt.matplotlib.dates.MonthLocator(interval=2))
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# 5. Subplots and Multiple Plots
print("\n5. SUBPLOTS AND MULTIPLE PLOTS:")

# Create a complex dashboard
fig = plt.figure(figsize=(16, 12))
```

```
# Main plot (large)
ax1 = plt.subplot2grid((3, 3), (0, 0), colspan=2, rowspan=2)
ax1.plot(x, y1, label='sin(x)', linewidth=2)
ax1.plot(x, y2, label='cos(x)', linewidth=2)
ax1.set_title('Main Plot', fontsize=14, fontweight='bold')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Small plots
ax2 = plt.subplot2grid((3, 3), (0, 2))
ax2.hist(data_hist, bins=20, alpha=0.7, color='skyblue')
ax2.set_title('Distribution')

ax3 = plt.subplot2grid((3, 3), (1, 2))
ax3.bar(categories, values, color='lightcoral')
ax3.set_title('Categories')

ax4 = plt.subplot2grid((3, 3), (2, 0), colspan=3)
ax4.scatter(x_scatter, y_scatter, alpha=0.6, c='green')
ax4.set_title('Scatter Plot')
ax4.set_xlabel('X')
ax4.set_ylabel('Y')

plt.suptitle('Data Visualization Dashboard', fontsize=16, fontweight='bol
plt.tight_layout()
plt.show()

# 6. Statistical Visualization
print("\n6. STATISTICAL VISUALIZATION:")

# Create sample data for statistical plots
np.random.seed(42)
data_stats = pd.DataFrame({
    'group': np.repeat(['A', 'B', 'C'], 50),
    'value': np.concatenate([
        np.random.normal(0, 1, 50),
        np.random.normal(1, 1.5, 50),
        np.random.normal(2, 0.8, 50)
    ]),
    'category': np.tile(['X', 'Y'], 75)
})

# Create statistical plots
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

# Distribution comparison
sns.histplot(data=data_stats, x='value', hue='group', kde=True, ax=axes[0
axes[0, 0].set_title('Distribution by Group')

# Box plot comparison
sns.boxplot(data=data_stats, x='group', y='value', hue='category', ax=axe
axes[0, 1].set_title('Box Plot by Group and Category')

# Violin plot
sns.violinplot(data=data_stats, x='group', y='value', ax=axes[1, 0])
axes[1, 0].set_title('Violin Plot by Group')

# Strip plot
sns.stripplot(data=data_stats, x='group', y='value', hue='category', ax=a
```

```
axes[1, 1].set_title('Strip Plot by Group and Category')

plt.tight_layout()
plt.show()

# 7. Export and Save Plots
print("\n7. EXPORT AND SAVE PLOTS:")

# Create a final plot to save
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, y1, label='sin(x)', linewidth=2)
ax.plot(x, y2, label='cos(x)', linewidth=2)
ax.set_title('Final Plot for Export')
ax.legend()
ax.grid(True, alpha=0.3)

# Save in different formats
plt.savefig('sample_plot.png', dpi=300, bbox_inches='tight')
plt.savefig('sample_plot.pdf', bbox_inches='tight')
plt.savefig('sample_plot.svg', bbox_inches='tight')

print("Plots saved as:")
print("- sample_plot.png (PNG format)")
print("- sample_plot.pdf (PDF format)")
print("- sample_plot.svg (SVG format)")

plt.show()

print("\n=== END OF VISUALIZATION REVIEW ===")
```

7. Comprehensive Machine Learning Review

7.1 Supervised Learning Algorithms

In [ ]:

```
# Comprehensive Machine Learning Review
print("=== COMPREHENSIVE MACHINE LEARNING REVIEW ===")

# Import additional ML libraries
from sklearn.linear_model import LogisticRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor, GradientBoostingClass
from sklearn.svm import SVC, SVR
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import GridSearchCV, cross_val_score
from sklearn.metrics import mean_squared_error, r2_score, confusion_matri

# 1. Data Preparation
print("\n1. DATA PREPARATION:")

# Create comprehensive dataset
np.random.seed(42)
n_samples = 1000

# Generate features
```

```
X = np.random.randn(n_samples, 5)
X[:, 0] = X[:, 0] * 2 + 1  # Make first feature more important
X[:, 1] = X[:, 1] * 1.5 + 0.5

# Generate target for regression
y_regression = 2 * X[:, 0] + 1.5 * X[:, 1] + 0.5 * X[:, 2] + np.random.ra

# Generate target for classification
y_classification = (X[:, 0] + X[:, 1] + np.random.randn(n_samples) * 0.5

# Create DataFrame
df_ml = pd.DataFrame(X, columns=['feature_1', 'feature_2', 'feature_3', '
df_ml['target_regression'] = y_regression
df_ml['target_classification'] = y_classification

print(f"Dataset shape: {df_ml.shape}")
print(f"Features: {df_ml.columns.tolist()[:5]}")
print(f"Regression target range: {df_ml['target_regression'].min():.2f} t
print(f"Classification target distribution: {df_ml['target_classification

# Split data
X_features = df_ml[['feature_1', 'feature_2', 'feature_3', 'feature_4', '
y_reg = df_ml['target_regression']
y_clf = df_ml['target_classification']

X_train, X_test, y_reg_train, y_reg_test = train_test_split(X_features, y
X_train_clf, X_test_clf, y_clf_train, y_clf_test = train_test_split(X_fea

print(f"\nTraining set size: {X_train.shape[0]}")
print(f"Test set size: {X_test.shape[0]}")

# 2. Regression Algorithms
print("\n2. REGRESSION ALGORITHMS:")

# Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_reg_train)
lr_pred = lr.predict(X_test)
lr_r2 = r2_score(y_reg_test, lr_pred)
lr_rmse = np.sqrt(mean_squared_error(y_reg_test, lr_pred))

print(f"Linear Regression - R²: {lr_r2:.3f}, RMSE: {lr_rmse:.3f}")

# Random Forest Regression
rf_reg = RandomForestRegressor(n_estimators=100, random_state=42)
rf_reg.fit(X_train, y_reg_train)
rf_reg_pred = rf_reg.predict(X_test)
rf_reg_r2 = r2_score(y_reg_test, rf_reg_pred)
rf_reg_rmse = np.sqrt(mean_squared_error(y_reg_test, rf_reg_pred))

print(f"Random Forest - R²: {rf_reg_r2:.3f}, RMSE: {rf_reg_rmse:.3f}")

# Ridge Regression
ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_reg_train)
ridge_pred = ridge.predict(X_test)
ridge_r2 = r2_score(y_reg_test, ridge_pred)
ridge_rmse = np.sqrt(mean_squared_error(y_reg_test, ridge_pred))

print(f"Ridge Regression - R²: {ridge_r2:.3f}, RMSE: {ridge_rmse:.3f}")
```

```python
# Support Vector Regression
svr = SVR(kernel='rbf')
svr.fit(X_train, y_reg_train)
svr_pred = svr.predict(X_test)
svr_r2 = r2_score(y_reg_test, svr_pred)
svr_rmse = np.sqrt(mean_squared_error(y_reg_test, svr_pred))

print(f"Support Vector Regression - R²: {svr_r2:.3f}, RMSE: {svr_rmse:.3f

# 3. Classification Algorithms
print("\n3. CLASSIFICATION ALGORITHMS:")

# Logistic Regression
log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train_clf, y_clf_train)
log_reg_pred = log_reg.predict(X_test_clf)
log_reg_acc = accuracy_score(y_clf_test, log_reg_pred)

print(f"Logistic Regression - Accuracy: {log_reg_acc:.3f}")

# Random Forest Classification
rf_clf = RandomForestClassifier(n_estimators=100, random_state=42)
rf_clf.fit(X_train_clf, y_clf_train)
rf_clf_pred = rf_clf.predict(X_test_clf)
rf_clf_acc = accuracy_score(y_clf_test, rf_clf_pred)

print(f"Random Forest - Accuracy: {rf_clf_acc:.3f}")

# Support Vector Machine
svm = SVC(kernel='rbf', random_state=42)
svm.fit(X_train_clf, y_clf_train)
svm_pred = svm.predict(X_test_clf)
svm_acc = accuracy_score(y_clf_test, svm_pred)

print(f"Support Vector Machine - Accuracy: {svm_acc:.3f}")

# K-Nearest Neighbors
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_clf, y_clf_train)
knn_pred = knn.predict(X_test_clf)
knn_acc = accuracy_score(y_clf_test, knn_pred)

print(f"K-Nearest Neighbors - Accuracy: {knn_acc:.3f}")

# Naive Bayes
nb = GaussianNB()
nb.fit(X_train_clf, y_clf_train)
nb_pred = nb.predict(X_test_clf)
nb_acc = accuracy_score(y_clf_test, nb_pred)

print(f"Naive Bayes - Accuracy: {nb_acc:.3f}")

# Decision Tree
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train_clf, y_clf_train)
dt_pred = dt.predict(X_test_clf)
dt_acc = accuracy_score(y_clf_test, dt_pred)

print(f"Decision Tree - Accuracy: {dt_acc:.3f}")
```

```python
# 4. Ensemble Methods
print("\n4. ENSEMBLE METHODS:")

# Gradient Boosting
gb = GradientBoostingClassifier(n_estimators=100, random_state=42)
gb.fit(X_train_clf, y_clf_train)
gb_pred = gb.predict(X_test_clf)
gb_acc = accuracy_score(y_clf_test, gb_pred)

print(f"Gradient Boosting - Accuracy: {gb_acc:.3f}")

# AdaBoost
ada = AdaBoostClassifier(n_estimators=100, random_state=42)
ada.fit(X_train_clf, y_clf_train)
ada_pred = ada.predict(X_test_clf)
ada_acc = accuracy_score(y_clf_test, ada_pred)

print(f"AdaBoost - Accuracy: {ada_acc:.3f}")

# Voting Classifier
voting_clf = VotingClassifier([
    ('lr', log_reg),
    ('rf', rf_clf),
    ('svm', svm)
], voting='hard')
voting_clf.fit(X_train_clf, y_clf_train)
voting_pred = voting_clf.predict(X_test_clf)
voting_acc = accuracy_score(y_clf_test, voting_pred)

print(f"Voting Classifier - Accuracy: {voting_acc:.3f}")

# 5. Unsupervised Learning
print("\n5. UNSUPERVISED LEARNING:")

# K-Means Clustering
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X_features)

print(f"K-Means clusters: {len(np.unique(kmeans_labels))}")
print(f"Cluster sizes: {np.bincount(kmeans_labels)}")

# Hierarchical Clustering
hierarchical = AgglomerativeClustering(n_clusters=3)
hierarchical_labels = hierarchical.fit_predict(X_features)

print(f"Hierarchical clusters: {len(np.unique(hierarchical_labels))}")
print(f"Cluster sizes: {np.bincount(hierarchical_labels)}")

# 6. Dimensionality Reduction
print("\n6. DIMENSIONALITY REDUCTION:")

# Principal Component Analysis
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_features)

print(f"PCA explained variance ratio: {pca.explained_variance_ratio_}")
print(f"Total variance explained: {pca.explained_variance_ratio_.sum():.3

# 7. Model Evaluation and Validation
```

```
print("\n7. MODEL EVALUATION AND VALIDATION:")

# Cross-validation
cv_scores = cross_val_score(rf_clf, X_train_clf, y_clf_train, cv=5)
print(f"Random Forest CV scores: {cv_scores}")
print(f"Mean CV score: {cv_scores.mean():.3f} (+/- {cv_scores.std() * 2:.

# Feature Importance
feature_importance = pd.DataFrame({
    'feature': X_features.columns,
    'importance': rf_clf.feature_importances_
}).sort_values('importance', ascending=False)

print(f"\nFeature Importance (Random Forest):")
print(feature_importance)

# 8. Hyperparameter Tuning
print("\n8. HYPERPARAMETER TUNING:")

# Grid Search for Random Forest
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10]
}

grid_search = GridSearchCV(RandomForestClassifier(random_state=42), param
grid_search.fit(X_train_clf, y_clf_train)

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.3f}")

# 9. Model Comparison
print("\n9. MODEL COMPARISON:")

# Create comparison DataFrame
models = ['Logistic Regression', 'Random Forest', 'SVM', 'KNN', 'Naive Ba
accuracies = [log_reg_acc, rf_clf_acc, svm_acc, knn_acc, nb_acc, dt_acc,

model_comparison = pd.DataFrame({
    'Model': models,
    'Accuracy': accuracies
}).sort_values('Accuracy', ascending=False)

print("Model Performance Comparison:")
print(model_comparison)

# 10. Visualization of Results
print("\n10. VISUALIZATION OF RESULTS:")

# Plot model comparison
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
plt.barh(model_comparison['Model'], model_comparison['Accuracy'])
plt.title('Model Accuracy Comparison')
plt.xlabel('Accuracy')

# Plot feature importance
plt.subplot(2, 2, 2)
```

```
plt.barh(feature_importance['feature'], feature_importance['importance'])
plt.title('Feature Importance (Random Forest)')
plt.xlabel('Importance')

# Plot PCA results
plt.subplot(2, 2, 3)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=kmeans_labels, alpha=0.6)
plt.title('PCA with K-Means Clustering')
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')

# Plot regression results
plt.subplot(2, 2, 4)
plt.scatter(y_reg_test, rf_reg_pred, alpha=0.6)
plt.plot([y_reg_test.min(), y_reg_test.max()], [y_reg_test.min(), y_reg_t
plt.title('Random Forest Regression Results')
plt.xlabel('Actual Values')
plt.ylabel('Predicted Values')

plt.tight_layout()
plt.show()

print("\n=== END OF MACHINE LEARNING REVIEW ===")
```

8. Project Planning and Execution Guidelines

8.1 Project Ideation and Planning

In [ ]:
```
# Project Planning and Execution Guidelines
print("=== PROJECT PLANNING AND EXECUTION GUIDELINES ===")

# 1. Project Ideation
print("\n1. PROJECT IDEATION:")
print("   - Choose a real-world problem")
print("   - Ensure data availability")
print("   - Define clear objectives")
print("   - Consider your skill level")

project_ideas = {
    "Beginner": [
        "Analyze sales data to identify trends",
        "Predict house prices using basic features",
        "Classify emails as spam/not spam",
        "Analyze customer demographics"
    ],
    "Intermediate": [
        "Predict stock prices using historical data",
        "Analyze social media sentiment",
        "Recommend products based on user behavior",
        "Analyze healthcare data for patterns"
    ],
    "Advanced": [
        "Build a recommendation system",
        "Analyze image data for classification",
        "Predict customer churn with complex features",
        "Analyze time series data for forecasting"
    ]
}
```

```
print("\nProject Ideas by Skill Level:")
for level, ideas in project_ideas.items():
    print(f"\n{level}:")
    for i, idea in enumerate(ideas, 1):
        print(f"  {i}. {idea}")

# 2. Project Planning Template
print("\n2. PROJECT PLANNING TEMPLATE:")

project_template = {
    "Project Title": "Your Project Name",
    "Problem Statement": "What problem are you solving?",
    "Objectives": [
        "Primary objective",
        "Secondary objectives"
    ],
    "Data Requirements": [
        "Data sources needed",
        "Data format and size",
        "Data quality requirements"
    ],
    "Methodology": [
        "Data collection approach",
        "Data preprocessing steps",
        "Analysis methods",
        "Model selection criteria"
    ],
    "Deliverables": [
        "Code repository",
        "Analysis report",
        "Visualizations",
        "Presentation"
    ],
    "Timeline": {
        "Week 1": "Data collection and exploration",
        "Week 2": "Data cleaning and preprocessing",
        "Week 3": "Model development and evaluation",
        "Week 4": "Results interpretation and presentation"
    }
}

print("\nProject Planning Template:")
for key, value in project_template.items():
    print(f"\n{key}:")
    if isinstance(value, list):
        for item in value:
            print(f"  - {item}")
    elif isinstance(value, dict):
        for k, v in value.items():
            print(f"  {k}: {v}")
    else:
        print(f"  {value}")

# 3. Data Science Workflow
print("\n3. DATA SCIENCE WORKFLOW:")

workflow_steps = [
    "1. Problem Definition",
    "2. Data Collection",
```

```
    "3. Data Exploration (EDA)",
    "4. Data Preprocessing",
    "5. Feature Engineering",
    "6. Model Selection",
    "7. Model Training",
    "8. Model Evaluation",
    "9. Model Deployment",
    "10. Results Communication"
]

print("\nComplete Data Science Workflow:")
for step in workflow_steps:
    print(f"  {step}")

# 4. Project Structure
print("\n4. RECOMMENDED PROJECT STRUCTURE:")

project_structure = """
my_data_science_project/
├── README.md
├── requirements.txt
├── environment.yml
├── data/
│   ├── raw/
│   ├── processed/
│   └── external/
├── notebooks/
│   ├── 01_data_exploration.ipynb
│   ├── 02_data_preprocessing.ipynb
│   ├── 03_model_development.ipynb
│   └── 04_results_analysis.ipynb
├── src/
│   ├── __init__.py
│   ├── data/
│   │   ├── __init__.py
│   │   └── make_dataset.py
│   ├── features/
│   │   ├── __init__.py
│   │   └── build_features.py
│   └── models/
│       ├── __init__.py
│       ├── train_model.py
│       └── predict_model.py
├── models/
├── reports/
│   ├── figures/
│   └── final_report.md
└── tests/
    ├── __init__.py
    └── test_data.py
"""

print(project_structure)

# 5. Data Collection Strategies
print("\n5. DATA COLLECTION STRATEGIES:")

data_sources = {
    "Public Datasets": [
        "Kaggle datasets",
```

```
        "UCI Machine Learning Repository",
        "Google Dataset Search",
        "Government open data"
    ],
    "APIs": [
        "Twitter API for social media data",
        "Financial APIs for stock data",
        "Weather APIs for climate data",
        "News APIs for text data"
    ],
    "Web Scraping": [
        "BeautifulSoup for HTML parsing",
        "Scrapy for large-scale scraping",
        "Selenium for dynamic content",
        "Requests for API calls"
    ],
    "Surveys and Forms": [
        "Google Forms",
        "SurveyMonkey",
        "Custom web forms",
        "Interview data"
    ]
}

print("\nData Collection Sources:")
for source_type, methods in data_sources.items():
    print(f"\n{source_type}:")
    for method in methods:
        print(f"  - {method}")

# 6. Project Execution Checklist
print("\n6. PROJECT EXECUTION CHECKLIST:")

checklist = {
    "Planning Phase": [
        "✓ Define clear project objectives",
        "✓ Identify data sources",
        "✓ Set up project structure",
        "✓ Create timeline and milestones"
    ],
    "Data Phase": [
        "✓ Collect and validate data",
        "✓ Perform exploratory data analysis",
        "✓ Clean and preprocess data",
        "✓ Document data quality issues"
    ],
    "Modeling Phase": [
        "✓ Select appropriate algorithms",
        "✓ Split data into train/validation/test",
        "✓ Train and tune models",
        "✓ Evaluate model performance"
    ],
    "Communication Phase": [
        "✓ Create compelling visualizations",
        "✓ Write clear documentation",
        "✓ Prepare presentation",
        "✓ Share results and insights"
    ]
}
```

```
print("\nProject Execution Checklist:")
for phase, tasks in checklist.items():
    print(f"\n{phase}:")
    for task in tasks:
        print(f"  {task}")

# 7. Common Pitfalls and Solutions
print("\n7. COMMON PITFALLS AND SOLUTIONS:")

pitfalls = {
    "Data Issues": {
        "Problem": "Poor data quality",
        "Solution": "Invest time in data cleaning and validation"
    },
    "Overfitting": {
        "Problem": "Model performs well on training but poorly on test da
        "Solution": "Use cross-validation and regularization techniques"
    },
    "Feature Engineering": {
        "Problem": "Not creating meaningful features",
        "Solution": "Domain knowledge and iterative feature creation"
    },
    "Evaluation": {
        "Problem": "Using inappropriate metrics",
        "Solution": "Choose metrics that align with business objectives"
    },
    "Documentation": {
        "Problem": "Poor documentation makes project hard to reproduce",
        "Solution": "Document every step and use version control"
    }
}

print("\nCommon Pitfalls and Solutions:")
for pitfall, details in pitfalls.items():
    print(f"\n{pitfall}:")
    print(f"  Problem: {details['Problem']}")
    print(f"  Solution: {details['Solution']}")

print("\n=== END OF PROJECT GUIDELINES ===")
```

9. Code Organization and Best Practices

9.1 Python Coding Standards

In [ ]:
```
# Code Organization and Best Practices
print("=== CODE ORGANIZATION AND BEST PRACTICES ===")

# 1. Python Coding Standards (PEP 8)
print("\n1. PYTHON CODING STANDARDS (PEP 8):")
print("   - Use 4 spaces for indentation")
print("   - Limit lines to 79 characters")
print("   - Use meaningful variable names")
print("   - Add docstrings to functions and classes")
print("   - Use blank lines to separate functions and classes")

# Example of good Python code
def calculate_customer_lifetime_value(monthly_revenue, churn_rate, months
    """
```

```
    Calculate customer lifetime value based on monthly revenue and churn

    Parameters:
    -----------
    monthly_revenue : float
        Average monthly revenue per customer
    churn_rate : float
        Monthly churn rate (between 0 and 1)
    months : int, optional
        Number of months to calculate for (default: 12)

    Returns:
    --------
    float
        Customer lifetime value
    """
    if churn_rate <= 0 or churn_rate >= 1:
        raise ValueError("Churn rate must be between 0 and 1")

    if monthly_revenue < 0:
        raise ValueError("Monthly revenue must be positive")

    # Calculate retention rate
    retention_rate = 1 - churn_rate

    # Calculate CLV
    clv = monthly_revenue * (retention_rate ** months) / (1 - retention_r

    return clv

# Example usage
clv = calculate_customer_lifetime_value(100, 0.05, 12)
print(f"Customer Lifetime Value: ${clv:.2f}")

# 2. Code Organization Principles
print("\n2. CODE ORGANIZATION PRINCIPLES:")

organization_principles = {
    "Single Responsibility": "Each function/class should do one thing wel
    "DRY (Don't Repeat Yourself)": "Avoid code duplication",
    "KISS (Keep It Simple, Stupid)": "Simple solutions are often better",
    "YAGNI (You Aren't Gonna Need It)": "Don't add features until needed"
    "Separation of Concerns": "Separate different aspects of functionalit
}

print("\nKey Principles:")
for principle, description in organization_principles.items():
    print(f"  {principle}: {description}")

# 3. Function Design Best Practices
print("\n3. FUNCTION DESIGN BEST PRACTICES:")

# Good function example
def process_sales_data(data, filter_date=None, group_by='month'):
    """
    Process sales data with optional filtering and grouping.

    Parameters:
    -----------
    data : pandas.DataFrame
```

```
        Sales data with columns: date, product, amount
    filter_date : str, optional
        Date to filter from (YYYY-MM-DD format)
    group_by : str, optional
        Grouping period ('day', 'month', 'year')

    Returns:
    --------
    pandas.DataFrame
        Processed sales data
    """
    # Input validation
    if not isinstance(data, pd.DataFrame):
        raise TypeError("Data must be a pandas DataFrame")

    required_columns = ['date', 'product', 'amount']
    if not all(col in data.columns for col in required_columns):
        raise ValueError(f"Data must contain columns: {required_columns}"

    # Create a copy to avoid modifying original data
    processed_data = data.copy()

    # Convert date column to datetime
    processed_data['date'] = pd.to_datetime(processed_data['date'])

    # Filter by date if specified
    if filter_date:
        filter_date = pd.to_datetime(filter_date)
        processed_data = processed_data[processed_data['date'] >= filter_

    # Group by specified period
    if group_by == 'day':
        processed_data['period'] = processed_data['date'].dt.date
    elif group_by == 'month':
        processed_data['period'] = processed_data['date'].dt.to_period('M
    elif group_by == 'year':
        processed_data['period'] = processed_data['date'].dt.to_period('Y
    else:
        raise ValueError("group_by must be 'day', 'month', or 'year'")

    # Aggregate data
    result = processed_data.groupby(['period', 'product'])['amount'].sum(

    return result

# 4. Error Handling Best Practices
print("\n4. ERROR HANDLING BEST PRACTICES:")

def safe_divide(a, b):
    """
    Safely divide two numbers with proper error handling.
    """
    try:
        result = a / b
        return result
    except ZeroDivisionError:
        print("Error: Division by zero")
        return None
    except TypeError:
        print("Error: Both arguments must be numbers")
```

```
        return None
    except Exception as e:
        print(f"Unexpected error: {e}")
        return None

# Test error handling
print(f"10 / 2 = {safe_divide(10, 2)}")
print(f"10 / 0 = {safe_divide(10, 0)}")
print(f"10 / 'a' = {safe_divide(10, 'a')}")

# 5. Data Processing Best Practices
print("\n5. DATA PROCESSING BEST PRACTICES:")

class DataProcessor:
    """
    A class for processing data with proper validation and error handling
    """

    def __init__(self, data):
        """
        Initialize the data processor.

        Parameters:
        -----------
        data : pandas.DataFrame
            Data to process
        """
        self.data = data.copy()
        self.validation_errors = []

    def validate_data(self):
        """
        Validate the data for common issues.

        Returns:
        --------
        bool
            True if data is valid, False otherwise
        """
        self.validation_errors = []

        # Check for empty DataFrame
        if self.data.empty:
            self.validation_errors.append("DataFrame is empty")

        # Check for missing values
        missing_values = self.data.isnull().sum()
        if missing_values.any():
            self.validation_errors.append(f"Missing values found: {missin

        # Check for duplicate rows
        duplicates = self.data.duplicated().sum()
        if duplicates > 0:
            self.validation_errors.append(f"Found {duplicates} duplicate

        return len(self.validation_errors) == 0

    def clean_data(self):
        """
        Clean the data by handling missing values and duplicates.
```

```
        """
        # Remove duplicates
        self.data = self.data.drop_duplicates()

        # Handle missing values (fill with median for numeric, mode for c
        for column in self.data.columns:
            if self.data[column].dtype in ['int64', 'float64']:
                self.data[column].fillna(self.data[column].median(), inpl
            else:
                self.data[column].fillna(self.data[column].mode()[0], inp

    def get_summary(self):
        """
        Get a summary of the data.

        Returns:
        --------
        dict
            Summary statistics
        """
        return {
            'shape': self.data.shape,
            'columns': list(self.data.columns),
            'dtypes': self.data.dtypes.to_dict(),
            'missing_values': self.data.isnull().sum().to_dict(),
            'numeric_summary': self.data.describe().to_dict() if not self
        }

# Example usage
sample_data = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'Alice', 'Diana'],
    'age': [25, 30, 35, 25, None],
    'salary': [50000, 60000, 70000, 50000, 55000],
    'department': ['IT', 'HR', 'IT', 'IT', 'Finance']
})

processor = DataProcessor(sample_data)
print(f"Data validation: {processor.validate_data()}")
if not processor.validate_data():
    print(f"Validation errors: {processor.validation_errors}")

processor.clean_data()
print(f"Cleaned data shape: {processor.data.shape}")
print(f"Data summary: {processor.get_summary()}")

# 6. Documentation Best Practices
print("\n6. DOCUMENTATION BEST PRACTICES:")

documentation_guidelines = {
    "Code Comments": [
        "Explain why, not what",
        "Use clear, concise language",
        "Update comments when code changes",
        "Remove outdated comments"
    ],
    "Docstrings": [
        "Use triple quotes for docstrings",
        "Include purpose, parameters, and return values",
        "Follow Google or NumPy docstring format",
        "Include examples for complex functions"
```

```
    ],
    "README Files": [
        "Include project description",
        "List installation instructions",
        "Provide usage examples",
        "Document dependencies"
    ],
    "Code Documentation": [
        "Use meaningful variable names",
        "Write self-documenting code",
        "Include type hints where appropriate",
        "Document complex algorithms"
    ]
}

print("\nDocumentation Guidelines:")
for category, guidelines in documentation_guidelines.items():
    print(f"\n{category}:")
    for guideline in guidelines:
        print(f"  - {guideline}")

# 7. Testing Best Practices
print("\n7. TESTING BEST PRACTICES:")

def test_data_processor():
    """
    Test the DataProcessor class.
    """
    # Test data
    test_data = pd.DataFrame({
        'value': [1, 2, 3, 4, 5],
        'category': ['A', 'B', 'A', 'B', 'A']
    })

    # Test initialization
    processor = DataProcessor(test_data)
    assert processor.data.shape == (5, 2), "Data shape should be (5, 2)"

    # Test validation
    assert processor.validate_data(), "Data should be valid"

    # Test summary
    summary = processor.get_summary()
    assert summary['shape'] == (5, 2), "Summary shape should be (5, 2)"

    print("All tests passed!")

# Run tests
test_data_processor()

# 8. Performance Best Practices
print("\n8. PERFORMANCE BEST PRACTICES:")

performance_tips = {
    "Data Processing": [
        "Use vectorized operations instead of loops",
        "Avoid iterating over DataFrames",
        "Use appropriate data types",
        "Consider memory usage for large datasets"
    ],
```

```
    "Code Optimization": [
        "Profile your code to find bottlenecks",
        "Use built-in functions when possible",
        "Avoid unnecessary computations",
        "Cache expensive operations"
    ],
    "Memory Management": [
        "Delete large objects when not needed",
        "Use generators for large datasets",
        "Consider chunking for large files",
        "Monitor memory usage"
    ]
}

print("\nPerformance Tips:")
for category, tips in performance_tips.items():
    print(f"\n{category}:")
    for tip in tips:
        print(f"  - {tip}")

print("\n=== END OF BEST PRACTICES ===")
```

10. Git Version Control and Collaboration

10.1 Git Fundamentals and Workflow

In [ ]:
```
# Git Version Control and Collaboration
print("=== GIT VERSION CONTROL AND COLLABORATION ===")

# 1. Git Fundamentals
print("\n1. GIT FUNDAMENTALS:")
print("   - Version control system for tracking changes")
print("   - Distributed version control")
print("   - Branching and merging capabilities")
print("   - Collaboration and backup")

git_concepts = {
    "Repository": "A project folder tracked by Git",
    "Commit": "A snapshot of changes at a point in time",
    "Branch": "A parallel version of the code",
    "Merge": "Combining changes from different branches",
    "Remote": "A copy of the repository on a server",
    "Clone": "Downloading a repository to local machine",
    "Push": "Uploading changes to remote repository",
    "Pull": "Downloading changes from remote repository"
}

print("\nKey Git Concepts:")
for concept, definition in git_concepts.items():
    print(f"  {concept}: {definition}")

# 2. Git Workflow
print("\n2. GIT WORKFLOW:")

workflow_steps = [
    "1. Initialize repository: git init",
    "2. Add files: git add .",
    "3. Commit changes: git commit -m 'message'",
```

```
    "4. Create branch: git branch feature-name",
    "5. Switch branch: git checkout feature-name",
    "6. Merge branch: git merge feature-name",
    "7. Push to remote: git push origin main",
    "8. Pull changes: git pull origin main"
]

print("\nBasic Git Workflow:")
for step in workflow_steps:
    print(f"  {step}")

# 3. Git Commands Reference
print("\n3. GIT COMMANDS REFERENCE:")

git_commands = {
    "Repository Setup": [
        "git init - Initialize new repository",
        "git clone <url> - Clone existing repository",
        "git remote add origin <url> - Add remote repository"
    ],
    "Basic Operations": [
        "git status - Check repository status",
        "git add <file> - Stage file for commit",
        "git add . - Stage all changes",
        "git commit -m 'message' - Commit changes",
        "git log - View commit history"
    ],
    "Branching": [
        "git branch - List branches",
        "git branch <name> - Create new branch",
        "git checkout <branch> - Switch to branch",
        "git merge <branch> - Merge branch",
        "git branch -d <branch> - Delete branch"
    ],
    "Remote Operations": [
        "git push origin <branch> - Push to remote",
        "git pull origin <branch> - Pull from remote",
        "git fetch - Download changes without merging",
        "git remote -v - List remote repositories"
    ]
}

print("\nGit Commands by Category:")
for category, commands in git_commands.items():
    print(f"\n{category}:")
    for command in commands:
        print(f"  {command}")

# 4. Git Best Practices
print("\n4. GIT BEST PRACTICES:")

git_best_practices = {
    "Commit Messages": [
        "Use clear, descriptive messages",
        "Start with capital letter",
        "Use present tense ('Add feature' not 'Added feature')",
        "Keep first line under 50 characters",
        "Add detailed description if needed"
    ],
    "Branching Strategy": [
```

```
        "Use feature branches for new features",
        "Keep main branch stable",
        "Use descriptive branch names",
        "Delete merged branches",
        "Use pull requests for code review"
    ],
    "File Management": [
        "Add .gitignore file",
        "Don't commit sensitive data",
        "Don't commit large files",
        "Commit related changes together",
        "Review changes before committing"
    ],
    "Collaboration": [
        "Pull before pushing",
        "Communicate with team members",
        "Use meaningful commit messages",
        "Resolve conflicts promptly",
        "Use pull requests for code review"
    ]
}

print("\nGit Best Practices:")
for category, practices in git_best_practices.items():
    print(f"\n{category}:")
    for practice in practices:
        print(f"  - {practice}")

# 5. .gitignore File
print("\n5. .GITIGNORE FILE:")

gitignore_content = """
# Python
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
*.egg-info/
.installed.cfg
*.egg

# Jupyter Notebook
.ipynb_checkpoints

# Environment variables
.env
.venv
```

```
env/
venv/
ENV/
env.bak/
venv.bak/

# IDE
.vscode/
.idea/
*.swp
*.swo

# OS
.DS_Store
.DS_Store?
._*
.Spotlight-V100
.Trashes
ehthumbs.db
Thumbs.db

# Data files
*.csv
*.xlsx
*.json
data/
*.pkl
*.h5

# Model files
models/
*.joblib
*.pkl

# Logs
*.log
logs/

# Temporary files
*.tmp
*.temp
"""

print("Sample .gitignore file:")
print(gitignore_content)

# 6. Collaboration Workflow
print("\n6. COLLABORATION WORKFLOW:")

collaboration_workflow = {
    "Fork and Clone": [
        "Fork the repository on GitHub",
        "Clone your fork to local machine",
        "Add upstream remote for original repository"
    ],
    "Development": [
        "Create feature branch from main",
        "Make changes and commit",
        "Push feature branch to your fork",
        "Create pull request to original repository"
```

```
    ],
    "Code Review": [
        "Review code changes",
        "Add comments and suggestions",
        "Approve or request changes",
        "Merge after approval"
    ],
    "Staying Updated": [
        "Fetch changes from upstream",
        "Merge or rebase with main branch",
        "Resolve any conflicts",
        "Update your fork"
    ]
}

print("\nCollaboration Workflow:")
for phase, steps in collaboration_workflow.items():
    print(f"\n{phase}:")
    for step in steps:
        print(f"  - {step}")

# 7. Git Hooks and Automation
print("\n7. GIT HOOKS AND AUTOMATION:")

git_hooks = {
    "Pre-commit": "Run tests and linting before commit",
    "Pre-push": "Run full test suite before pushing",
    "Post-commit": "Send notifications or update documentation",
    "Pre-receive": "Server-side validation before accepting pushes"
}

print("\nGit Hooks:")
for hook, purpose in git_hooks.items():
    print(f"  {hook}: {purpose}")

# 8. Common Git Issues and Solutions
print("\n8. COMMON GIT ISSUES AND SOLUTIONS:")

git_issues = {
    "Merge Conflicts": {
        "Problem": "Conflicting changes in same file",
        "Solution": "Edit file to resolve conflicts, then commit"
    },
    "Accidental Commit": {
        "Problem": "Committed wrong changes",
        "Solution": "Use git reset or git revert"
    },
    "Lost Commits": {
        "Problem": "Can't find previous commits",
        "Solution": "Use git reflog to find lost commits"
    },
    "Large Files": {
        "Problem": "Trying to commit large files",
        "Solution": "Use Git LFS or add to .gitignore"
    },
    "Wrong Branch": {
        "Problem": "Committed to wrong branch",
        "Solution": "Use git cherry-pick to move commits"
    }
}
```

```
print("\nCommon Git Issues and Solutions:")
for issue, details in git_issues.items():
    print(f"\n{issue}:")
    print(f"  Problem: {details['Problem']}")
    print(f"  Solution: {details['Solution']}")

# 9. Git Integration with Data Science
print("\n9. GIT INTEGRATION WITH DATA SCIENCE:")

ds_git_tips = {
    "Notebook Management": [
        "Clear output before committing notebooks",
        "Use nbstripout to remove outputs",
        "Consider converting to .py files for code",
        "Use git LFS for large notebooks"
    ],
    "Data Management": [
        "Never commit large datasets",
        "Use .gitignore for data files",
        "Document data sources and processing",
        "Use DVC for data version control"
    ],
    "Model Management": [
        "Don't commit model files",
        "Document model parameters",
        "Use MLflow for experiment tracking",
        "Version control model code only"
    ],
    "Environment Management": [
        "Commit requirements.txt",
        "Use conda environment.yml",
        "Document Python version",
        "Use Docker for reproducibility"
    ]
}

print("\nData Science Git Tips:")
for category, tips in ds_git_tips.items():
    print(f"\n{category}:")
    for tip in tips:
        print(f"  - {tip}")

# 10. Git Commands for This Project
print("\n10. GIT COMMANDS FOR THIS PROJECT:")

project_git_commands = [
    "# Initialize repository",
    "git init",
    "",
    "# Add all files",
    "git add .",
    "",
    "# First commit",
    "git commit -m 'Initial commit: Python course materials'",
    "",
    "# Add remote repository (replace with your GitHub repo)",
    "git remote add origin https://github.com/yourusername/python-course.
    "",
    "# Push to GitHub",
```

```
    "git push -u origin main",
    "",
    "# Create feature branch for new module",
    "git checkout -b feature/new-module",
    "",
    "# After making changes",
    "git add .",
    "git commit -m 'Add new module content'",
    "git push origin feature/new-module",
    "",
    "# Merge back to main",
    "git checkout main",
    "git merge feature/new-module",
    "git push origin main"
]

print("\nGit Commands for This Project:")
for command in project_git_commands:
    print(command)

print("\n=== END OF GIT WORKFLOW ===")
```

11. Future Learning Paths and Advanced Topics

11.1 Advanced Python and Data Science

In [ ]:
```
# Future Learning Paths and Advanced Topics
print("=== FUTURE LEARNING PATHS AND ADVANCED TOPICS ===")

# 1. Advanced Python Topics
print("\n1. ADVANCED PYTHON TOPICS:")

advanced_python = {
    "Object-Oriented Programming": [
        "Advanced class design patterns",
        "Metaclasses and descriptors",
        "Multiple inheritance and mixins",
        "Property decorators and getters/setters"
    ],
    "Functional Programming": [
        "Lambda functions and closures",
        "Decorators and function composition",
        "Generators and iterators",
        "Map, filter, and reduce operations"
    ],
    "Concurrency and Parallelism": [
        "Threading and multiprocessing",
        "Asyncio for asynchronous programming",
        "Parallel processing with joblib",
        "GPU computing with CuPy"
    ],
    "Performance Optimization": [
        "Profiling and benchmarking",
        "Cython for speed optimization",
        "NumPy vectorization techniques",
        "Memory optimization strategies"
    ]
}
```

```python
print("\nAdvanced Python Topics:")
for category, topics in advanced_python.items():
    print(f"\n{category}:")
    for topic in topics:
        print(f"  - {topic}")

# 2. Advanced Data Science Libraries
print("\n2. ADVANCED DATA SCIENCE LIBRARIES:")

advanced_libraries = {
    "Data Processing": [
        "Dask - Parallel computing for larger datasets",
        "Vaex - Out-of-core DataFrames",
        "Polars - Fast DataFrame library",
        "Modin - Pandas on Ray for scaling"
    ],
    "Machine Learning": [
        "XGBoost - Gradient boosting framework",
        "LightGBM - Light gradient boosting",
        "CatBoost - Categorical boosting",
        "Optuna - Hyperparameter optimization"
    ],
    "Deep Learning": [
        "TensorFlow - Google's deep learning framework",
        "PyTorch - Facebook's deep learning framework",
        "Keras - High-level neural network API",
        "Transformers - Hugging Face NLP models"
    ],
    "Visualization": [
        "Plotly - Interactive visualizations",
        "Bokeh - Interactive web visualizations",
        "Altair - Grammar of graphics",
        "Dash - Web applications for data science"
    ]
}

print("\nAdvanced Data Science Libraries:")
for category, libraries in advanced_libraries.items():
    print(f"\n{category}:")
    for library in libraries:
        print(f"  - {library}")

# 3. Specialized Data Science Domains
print("\n3. SPECIALIZED DATA SCIENCE DOMAINS:")

specialized_domains = {
    "Natural Language Processing": [
        "Text preprocessing and tokenization",
        "Sentiment analysis and text classification",
        "Named entity recognition",
        "Language models and transformers"
    ],
    "Computer Vision": [
        "Image preprocessing and augmentation",
        "Object detection and recognition",
        "Image segmentation",
        "Convolutional neural networks"
    ],
    "Time Series Analysis": [
```

```
        "ARIMA and seasonal decomposition",
        "Prophet for forecasting",
        "LSTM networks for time series",
        "Anomaly detection in time series"
    ],
    "Recommendation Systems": [
        "Collaborative filtering",
        "Content-based filtering",
        "Matrix factorization",
        "Deep learning for recommendations"
    ]
}

print("\nSpecialized Data Science Domains:")
for domain, topics in specialized_domains.items():
    print(f"\n{domain}:")
    for topic in topics:
        print(f"  - {topic}")

# 4. Big Data and Cloud Computing
print("\n4. BIG DATA AND CLOUD COMPUTING:")

big_data_cloud = {
    "Big Data Processing": [
        "Apache Spark with PySpark",
        "Hadoop ecosystem",
        "Apache Kafka for streaming",
        "Dask for distributed computing"
    ],
    "Cloud Platforms": [
        "AWS (S3, EC2, SageMaker)",
        "Google Cloud Platform",
        "Microsoft Azure",
        "Databricks platform"
    ],
    "Containerization": [
        "Docker for containerization",
        "Kubernetes for orchestration",
        "JupyterHub for multi-user environments",
        "MLflow for experiment tracking"
    ],
    "Data Engineering": [
        "Apache Airflow for workflows",
        "ETL/ELT pipelines",
        "Data warehousing concepts",
        "Real-time data processing"
    ]
}

print("\nBig Data and Cloud Computing:")
for category, topics in big_data_cloud.items():
    print(f"\n{category}:")
    for topic in topics:
        print(f"  - {topic}")

# 5. Production and Deployment
print("\n5. PRODUCTION AND DEPLOYMENT:")

production_deployment = {
    "Model Deployment": [
```

```
        "REST APIs with Flask/FastAPI",
        "Model serving with TensorFlow Serving",
        "Containerized deployments",
        "A/B testing for models"
    ],
    "Monitoring and Maintenance": [
        "Model performance monitoring",
        "Data drift detection",
        "Model retraining pipelines",
        "MLOps best practices"
    ],
    "Scalability": [
        "Microservices architecture",
        "Load balancing and scaling",
        "Caching strategies",
        "Database optimization"
    ],
    "Security and Privacy": [
        "Data privacy and GDPR",
        "Model security and adversarial attacks",
        "Encryption and secure communication",
        "Ethical AI considerations"
    ]
}

print("\nProduction and Deployment:")
for category, topics in production_deployment.items():
    print(f"\n{category}:")
    for topic in topics:
        print(f"  - {topic}")

# 6. Learning Resources and Certifications
print("\n6. LEARNING RESOURCES AND CERTIFICATIONS:")

learning_resources = {
    "Online Courses": [
        "Coursera - Machine Learning Specialization",
        "edX - MIT Introduction to Machine Learning",
        "Udacity - Data Science Nanodegree",
        "Fast.ai - Practical Deep Learning"
    ],
    "Books": [
        "Hands-On Machine Learning by Aurélien Géron",
        "Python for Data Analysis by Wes McKinney",
        "The Elements of Statistical Learning",
        "Pattern Recognition and Machine Learning"
    ],
    "Certifications": [
        "Google Cloud Professional Data Engineer",
        "AWS Certified Machine Learning Specialty",
        "Microsoft Azure Data Scientist Associate",
        "IBM Data Science Professional Certificate"
    ],
    "Communities": [
        "Kaggle - Competitions and datasets",
        "Stack Overflow - Q&A community",
        "Reddit r/MachineLearning",
        "Data Science Meetups and conferences"
    ]
}
```

```
print("\nLearning Resources and Certifications:")
for category, resources in learning_resources.items():
    print(f"\n{category}:")
    for resource in resources:
        print(f"  - {resource}")

# 7. Career Paths in Data Science
print("\n7. CAREER PATHS IN DATA SCIENCE:")

career_paths = {
    "Data Analyst": {
        "Skills": ["SQL", "Excel", "Tableau", "Statistics"],
        "Responsibilities": ["Data analysis", "Reporting", "Dashboard cre
        "Salary Range": "$50,000 - $80,000"
    },
    "Data Scientist": {
        "Skills": ["Python/R", "Machine Learning", "Statistics", "SQL"],
        "Responsibilities": ["Model building", "Data analysis", "Research
        "Salary Range": "$80,000 - $120,000"
    },
    "Machine Learning Engineer": {
        "Skills": ["Python", "ML/DL", "Software Engineering", "Cloud"],
        "Responsibilities": ["Model deployment", "ML pipelines", "Infrast
        "Salary Range": "$100,000 - $150,000"
    },
    "Data Engineer": {
        "Skills": ["Python/Java", "Big Data", "ETL", "Cloud"],
        "Responsibilities": ["Data pipelines", "Infrastructure", "Data wa
        "Salary Range": "$90,000 - $130,000"
    }
}

print("\nCareer Paths in Data Science:")
for role, details in career_paths.items():
    print(f"\n{role}:")
    for key, value in details.items():
        print(f"  {key}: {value}")

# 8. Recommended Learning Timeline
print("\n8. RECOMMENDED LEARNING TIMELINE:")

learning_timeline = {
    "Month 1-2": [
        "Master Python fundamentals",
        "Learn NumPy and Pandas",
        "Practice with real datasets",
        "Complete basic projects"
    ],
    "Month 3-4": [
        "Learn data visualization",
        "Study statistics and probability",
        "Practice exploratory data analysis",
        "Work on intermediate projects"
    ],
    "Month 5-6": [
        "Learn machine learning algorithms",
        "Practice model building",
        "Study evaluation metrics",
        "Complete ML projects"
```

```
    ],
    "Month 7-8": [
        "Learn advanced ML techniques",
        "Study deep learning basics",
        "Practice with larger datasets",
        "Work on portfolio projects"
    ],
    "Month 9-12": [
        "Specialize in chosen domain",
        "Learn deployment and production",
        "Contribute to open source",
        "Prepare for job applications"
    ]
}

print("\nRecommended Learning Timeline:")
for period, activities in learning_timeline.items():
    print(f"\n{period}:")
    for activity in activities:
        print(f"  - {activity}")

# 9. Project Ideas for Portfolio
print("\n9. PROJECT IDEAS FOR PORTFOLIO:")

portfolio_projects = {
    "Beginner Projects": [
        "Analyze COVID-19 data trends",
        "Predict house prices",
        "Customer segmentation analysis",
        "Sentiment analysis of social media"
    ],
    "Intermediate Projects": [
        "Build a recommendation system",
        "Predict stock prices",
        "Image classification with CNN",
        "Time series forecasting"
    ],
    "Advanced Projects": [
        "End-to-end ML pipeline",
        "Real-time data processing",
        "Deep learning model deployment",
        "Multi-modal data analysis"
    ],
    "Specialized Projects": [
        "NLP chatbot development",
        "Computer vision application",
        "Fraud detection system",
        "Predictive maintenance model"
    ]
}

print("\nProject Ideas for Portfolio:")
for level, projects in portfolio_projects.items():
    print(f"\n{level}:")
    for project in projects:
        print(f"  - {project}")

# 10. Staying Updated in Data Science
print("\n10. STAYING UPDATED IN DATA SCIENCE:")
```

```
staying_updated = {
    "News and Blogs": [
        "Towards Data Science on Medium",
        "KDnuggets",
        "Data Science Central",
        "Analytics Vidhya"
    ],
    "Research Papers": [
        "arXiv.org for latest research",
        "Google Scholar for citations",
        "Papers with Code for implementations",
        "Conference proceedings (NeurIPS, ICML)"
    ],
    "Podcasts": [
        "Data Skeptic",
        "Not So Standard Deviations",
        "The Data Show",
        "Linear Digressions"
    ],
    "Conferences": [
        "PyData conferences",
        "Strata Data Conference",
        "ODSC (Open Data Science Conference)",
        "Local meetups and workshops"
    ]
}

print("\nStaying Updated in Data Science:")
for category, sources in staying_updated.items():
    print(f"\n{category}:")
    for source in sources:
        print(f"  - {source}")

print("\n=== END OF FUTURE LEARNING PATHS ===")
```

12. Final Project Guidelines and Next Steps

12.1 Your Final Project Assignment

In [ ]:
```
# Final Project Guidelines and Next Steps
print("=== FINAL PROJECT GUIDELINES AND NEXT STEPS ===")

# 1. Final Project Assignment
print("\n1. FINAL PROJECT ASSIGNMENT:")
print("   Congratulations! You've completed the comprehensive Python cour
print("   Now it's time to apply everything you've learned in a real-worl

project_requirements = {
    "Project Scope": [
        "Choose a real-world dataset (minimum 1000 rows)",
        "Apply complete data science workflow",
        "Include data cleaning, analysis, and modeling",
        "Create compelling visualizations",
        "Document your process thoroughly"
    ],
    "Technical Requirements": [
        "Use Python, NumPy, Pandas, and scikit-learn",
        "Include at least 3 different ML algorithms",
```

```
        "Create at least 5 different visualizations",
        "Implement proper data preprocessing",
        "Include model evaluation and comparison"
    ],
    "Deliverables": [
        "Jupyter notebook with complete analysis",
        "Clean, well-documented Python code",
        "Executive summary of findings",
        "Presentation slides (5-10 minutes)",
        "GitHub repository with all files"
    ],
    "Timeline": [
        "Week 1: Project planning and data collection",
        "Week 2: Data exploration and preprocessing",
        "Week 3: Model development and evaluation",
        "Week 4: Documentation and presentation"
    ]
}

print("\nProject Requirements:")
for category, requirements in project_requirements.items():
    print(f"\n{category}:")
    for requirement in requirements:
        print(f"  - {requirement}")

# 2. Project Evaluation Criteria
print("\n2. PROJECT EVALUATION CRITERIA:")

evaluation_criteria = {
    "Technical Skills (40%)": [
        "Code quality and organization",
        "Proper use of Python libraries",
        "Data preprocessing techniques",
        "Model selection and evaluation",
        "Visualization effectiveness"
    ],
    "Analysis Quality (30%)": [
        "Data exploration depth",
        "Statistical analysis rigor",
        "Insight generation",
        "Problem-solving approach",
        "Results interpretation"
    ],
    "Communication (20%)": [
        "Clear documentation",
        "Effective visualizations",
        "Presentation skills",
        "Executive summary quality",
        "Code comments and docstrings"
    ],
    "Creativity (10%)": [
        "Unique insights",
        "Creative visualizations",
        "Innovative approaches",
        "Real-world applicability",
        "Problem selection"
    ]
}

print("\nEvaluation Criteria:")
```

```python
for category, criteria in evaluation_criteria.items():
    print(f"\n{category}:")
    for criterion in criteria:
        print(f"  - {criterion}")

# 3. Suggested Project Ideas
print("\n3. SUGGESTED PROJECT IDEAS:")

suggested_projects = {
    "Business Analytics": [
        "Customer churn prediction for telecom company",
        "Sales forecasting for retail business",
        "Market basket analysis for e-commerce",
        "Employee performance prediction"
    ],
    "Healthcare": [
        "Disease prediction from patient data",
        "Drug effectiveness analysis",
        "Hospital readmission prediction",
        "Medical image classification"
    ],
    "Finance": [
        "Credit risk assessment",
        "Stock price prediction",
        "Fraud detection in transactions",
        "Cryptocurrency analysis"
    ],
    "Social Impact": [
        "Climate change data analysis",
        "Education outcome prediction",
        "Crime pattern analysis",
        "Social media sentiment analysis"
    ]
}

print("\nSuggested Project Ideas:")
for domain, projects in suggested_projects.items():
    print(f"\n{domain}:")
    for project in projects:
        print(f"  - {project}")

# 4. Project Template Structure
print("\n4. PROJECT TEMPLATE STRUCTURE:")

project_template = """
# Your Project Title

## 1. Executive Summary
- Problem statement
- Key findings
- Recommendations

## 2. Data Collection and Description
- Data source
- Dataset description
- Data quality assessment

## 3. Exploratory Data Analysis
- Data overview
- Statistical summaries
```

```
- Visualizations
- Key insights

## 4. Data Preprocessing
- Missing value handling
- Feature engineering
- Data transformation
- Train/test split

## 5. Model Development
- Algorithm selection
- Model training
- Hyperparameter tuning
- Cross-validation

## 6. Model Evaluation
- Performance metrics
- Model comparison
- Feature importance
- Validation results

## 7. Results and Insights
- Key findings
- Business implications
- Limitations
- Future work

## 8. Conclusion
- Summary
- Recommendations
- Next steps
"""

print("Project Template Structure:")
print(project_template)

# 5. Next Steps After Course Completion
print("\n5. NEXT STEPS AFTER COURSE COMPLETION:")

next_steps = {
    "Immediate Actions": [
        "Complete your final project",
        "Create a GitHub portfolio",
        "Write a technical blog post",
        "Share your work on LinkedIn"
    ],
    "Skill Development": [
        "Learn advanced ML techniques",
        "Study deep learning",
        "Practice with larger datasets",
        "Learn cloud platforms"
    ],
    "Career Preparation": [
        "Build a strong portfolio",
        "Network with professionals",
        "Apply for internships/jobs",
        "Consider certifications"
    ],
    "Continuous Learning": [
        "Follow data science blogs",
```

```
        "Join online communities",
        "Attend conferences/meetups",
        "Contribute to open source"
    ]
}

print("\nNext Steps After Course Completion:")
for category, steps in next_steps.items():
    print(f"\n{category}:")
    for step in steps:
        print(f"  - {step}")

# 6. Resources for Continued Learning
print("\n6. RESOURCES FOR CONTINUED LEARNING:")

continued_learning = {
    "Advanced Courses": [
        "Fast.ai - Practical Deep Learning",
        "Coursera - Deep Learning Specialization",
        "edX - MIT Introduction to Machine Learning",
        "Udacity - Machine Learning Engineer Nanodegree"
    ],
    "Practice Platforms": [
        "Kaggle - Competitions and datasets",
        "HackerRank - Coding challenges",
        "LeetCode - Algorithm problems",
        "DataCamp - Interactive courses"
    ],
    "Books": [
        "Hands-On Machine Learning by Aurélien Géron",
        "Python for Data Analysis by Wes McKinney",
        "The Elements of Statistical Learning",
        "Pattern Recognition and Machine Learning"
    ],
    "Communities": [
        "Reddit r/MachineLearning",
        "Stack Overflow",
        "GitHub",
        "LinkedIn Data Science groups"
    ]
}

print("\nResources for Continued Learning:")
for category, resources in continued_learning.items():
    print(f"\n{category}:")
    for resource in resources:
        print(f"  - {resource}")

# 7. Final Words of Encouragement
print("\n7. FINAL WORDS OF ENCOURAGEMENT:")
print("""
Congratulations on completing this comprehensive Python course! 🎉

You've learned:
✓ Python programming fundamentals
✓ Data manipulation with NumPy and Pandas
✓ Data visualization with Matplotlib and Seaborn
✓ Machine learning with scikit-learn
✓ Best practices for data science projects
✓ Version control with Git
```

```
✓ Project planning and execution

Remember:
- Data science is a journey, not a destination
- Practice makes perfect - keep coding!
- Don't be afraid to make mistakes - they're learning opportunities
- Stay curious and keep exploring new techniques
- Share your knowledge with others

Your final project is your chance to showcase everything you've learned.
Make it count! 💪

Good luck with your data science journey!
""")

print("\n=== END OF MODULE 10 ===")
print("=== CONGRATULATIONS ON COMPLETING THE COURSE! ===")
```

In [ ]:

```
# Step 1: Exploratory Data Analysis
print("=== EXPLORATORY DATA ANALYSIS ===")
print(f"Dataset shape: {df.shape}")
print(f"\nFirst 5 rows:")
print(df.head())

print(f"\nBasic statistics:")
print(df.describe())

print(f"\nMissing values:")
print(df.isnull().sum())

# Step 2: Data Visualization
plt.figure(figsize=(15, 10))

# Subplot 1: Age distribution
plt.subplot(2, 3, 1)
plt.hist(df['age'], bins=20, alpha=0.7, color='skyblue')
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Frequency')

# Subplot 2: Income vs Salary scatter
plt.subplot(2, 3, 2)
plt.scatter(df['income'], df['salary'], alpha=0.6, color='green')
plt.title('Income vs Salary')
plt.xlabel('Income')
plt.ylabel('Salary')

# Subplot 3: Education distribution
plt.subplot(2, 3, 3)
df['education'].value_counts().plot(kind='bar', color='orange')
plt.title('Education Distribution')
plt.xlabel('Education Level')
plt.ylabel('Count')
plt.xticks(rotation=45)

# Subplot 4: Experience vs Salary
plt.subplot(2, 3, 4)
plt.scatter(df['experience'], df['salary'], alpha=0.6, color='red')
plt.title('Experience vs Salary')
plt.xlabel('Experience (years)')
```

```
plt.ylabel('Salary')

# Subplot 5: Correlation heatmap
plt.subplot(2, 3, 5)
numeric_cols = df.select_dtypes(include=[np.number]).columns
correlation_matrix = df[numeric_cols].corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Matrix')

# Subplot 6: Box plot for salary by education
plt.subplot(2, 3, 6)
df.boxplot(column='salary', by='education', ax=plt.gca())
plt.title('Salary by Education Level')
plt.xlabel('Education Level')
plt.ylabel('Salary')

plt.tight_layout()
plt.show()

# Step 3: Data Preprocessing
print("\n=== DATA PREPROCESSING ===")

# Create binary classification target (high salary > 70000)
df['high_salary'] = (df['salary'] > 70000).astype(int)
print(f"High salary distribution:")
print(df['high_salary'].value_counts())

# Prepare features for ML
X = df[['age', 'income', 'experience']]
y = df['high_salary']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
print(f"Training set size: {X_train.shape[0]}")
print(f"Test set size: {X_test.shape[0]}")

# Step 4: Machine Learning Model
print("\n=== MACHINE LEARNING MODEL ===")

# Train Random Forest Classifier
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Make predictions
y_pred = rf_model.predict(X_test)

# Evaluate model
accuracy = accuracy_score(y_test, y_pred)
print(f"Model Accuracy: {accuracy:.2f}")

print(f"\nClassification Report:")
print(classification_report(y_test, y_pred))

# Feature importance
feature_importance = pd.DataFrame({
    'feature': X.columns,
    'importance': rf_model.feature_importances_
}).sort_values('importance', ascending=False)
```

```
print(f"\nFeature Importance:")
print(feature_importance)
```